FILE EXPORTATION STEPS

- **Step 1:** Import the required Libraries.
- **Step 2:** Get the Absolute path of the current working Directory (Which contains our raw files) and Change Directory
- **Step 3:** Get the absolute path of all our Csv files using **glob** library
- **Step 4:** Loop through one file at a time and read it into a data frame using **pandas**
- **Step 5:** Change the data columns to the desired datatypes for Usuability
- **Step 6:** Connect to the SQL data base and Push the raw file

Step 1: Import libraries

In [13]:
```
#Libraries
from sqlalchemy import create_engine
import pyodbc
import pandas as pd
import numpy as np
import glob
import os
import urllib
```

In [ ]:
```
pip install pyodbc
```

- **sqlalchemy(create_engine):** - Acts as a translator; Provides connectivity between SQL and Python
  - Creates an engine that allows us send tables directly to our database without use of long SQL INSERT statements
- **pyodbc(PythonOpenDatabaseConnection):** - Bridge/driver that allows python to physically connect to SQLserver database, Low-level handshake between Computer and Database Server.
- **pandas:** - For reading csv files into dataframes(Structured tables)
- **numpy:**- Handling Division by Zero in Population(#DIV/0!), mathematical computations. To handle NAN values.
- **glob:** - File finder. that allows us to search for files using wild cards like *.csv insteading of typing them manually.
- **OS:** - Allows python to talk to windows operating system. to tell our current working directory pathname
- **urllib:** - Converts cryptic characters into format that the **create_engine** function can understand. Encoding Connection string

Step II: Understand the Absolute path of our raw files

In [2]:
```
#Find out our current Working Directory
os.getcwd()
```

Out[2]:
```
'D:\\Data Analyst\\MALARIA ANALYTICS\\FULL PROJECT\\Malaria data\\Raw data(Origina
l Data)'
```

In [3]:
```
#Change Directory to our current  Working Directory
os.chdir(
'D:\\Data Analyst\\MALARIA ANALYTICS\\FULL PROJECT\\Malaria data\\Raw data(Original
```

In [21]:
```
#Fetch the Absolute paths for our individual raw files.
glob.glob(
'D:\\Data Analyst\\MALARIA ANALYTICS\\FULL PROJECT\\Malaria data\\Raw data(Original
```

Out[21]:
```
['D:\\Data Analyst\\MALARIA ANALYTICS\\FULL PROJECT\\Malaria data\\Raw data(Origin
al Data)/Raw data\\Jamir Data 2020.csv',
 'D:\\Data Analyst\\MALARIA ANALYTICS\\FULL PROJECT\\Malaria data\\Raw data(Origin
al Data)/Raw data\\Jamir Data 2021.csv',
 'D:\\Data Analyst\\MALARIA ANALYTICS\\FULL PROJECT\\Malaria data\\Raw data(Origin
al Data)/Raw data\\Jamir Data 2022.csv',
 'D:\\Data Analyst\\MALARIA ANALYTICS\\FULL PROJECT\\Malaria data\\Raw data(Origin
al Data)/Raw data\\Jamir Data 2023.csv',
 'D:\\Data Analyst\\MALARIA ANALYTICS\\FULL PROJECT\\Malaria data\\Raw data(Origin
al Data)/Raw data\\Jamir Data 2024.csv']
```

Step III: Loop and Read the files into dataframes (One at a time)

In [22]:
```
all_files = glob.glob('D:\\Data Analyst\\MALARIA ANALYTICS\\FULL PROJECT\\Malaria d
dfs = []
for file in all_files:
    df1 = pd.read_csv(file)
```

In [23]:
```
#DataType information
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 161 entries, 0 to 160
Columns: 487 entries, orgunitlevel1 to Population
dtypes: float64(197), int64(284), object(6)
memory usage: 612.7+ KB
```

In [24]:
```
#Number of Rows and Columns
df1.shape
```

Out[24]:
```
(161, 487)
```

In [19]:
```
#First 5 records of dataset
df1.head()
```

Out[19]:

	Region	District	0-28 DAYS FEMALE	0-28 DAYS MALE	0-28 DAYS TOTAL	10-19 YEARS FEMALE	10-19 YEARS MALE	10-19 YRS TOTAL	20+ YRS FEMALE	M
0	Karamoja	Abim District	2.0	5.0	7.0	13939.0	6695.0	20634.0	15856.0	7
1	Acholi	Acholi	368.0	310.0	678.0	246462.0	132169.0	378631.0	228667.0	82
2	West Nile	Adjumani District	24.0	15.0	39.0	63279.0	44827.0	108106.0	60259.0	26
3	Acholi	Agago District	25.0	21.0	46.0	40202.0	20974.0	61176.0	38604.0	11
4	Lango	Alebtong District	50.0	32.0	82.0	18176.0	8163.0	26339.0	24600.0	6

5 rows × 65 columns

Step IV: Change columns to respective data types

In [7]:

```
#Change Numeric Column Values to Integer
for col in df1.columns:
    if pd.api.types.is_numeric_dtype(df1[col]):
        # If all non-missing values are whole numbers, cast to Int64
        if (df1[col].dropna() % 1 == 0).all():
            df1[col] = df1[col].astype("Int64")   # nullable integer
        else:
            # Otherwise keep as float (decimals preserved)
            df1[col] = df1[col].astype("float")
    else:
        # Try to coerce text columns that look numeric, leave others untouched
        df1[col] = pd.to_numeric(df1[col], errors="ignore")
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_5540\1702007171.py:12: FutureWarning: err
ors='ignore' is deprecated and will raise in a future version. Use to_numeric withou
t passing `errors` and catch exceptions explicitly instead
  df1[col] = pd.to_numeric(df1[col], errors="ignore")
C:\Users\User\AppData\Local\Temp\ipykernel_5540\1702007171.py:12: FutureWarning: err
ors='ignore' is deprecated and will raise in a future version. Use to_numeric withou
t passing `errors` and catch exceptions explicitly instead
  df1[col] = pd.to_numeric(df1[col], errors="ignore")
C:\Users\User\AppData\Local\Temp\ipykernel_5540\1702007171.py:12: FutureWarning: err
ors='ignore' is deprecated and will raise in a future version. Use to_numeric withou
t passing `errors` and catch exceptions explicitly instead
  df1[col] = pd.to_numeric(df1[col], errors="ignore")
```

In [27]:

```
# Replace the specific error string with NaN, then convert
df1['Population'] = df1['Population'].replace('#DIV/0!', np.nan).astype(float)
```

In [ ]:
```
#Dataframe Inspection
df1.head()
```

Step V: Push the raw files to the SQL server Database

In [20]:
```
#Step 1: Define the Server instance and Database Name
server = "DESKTOP-U507UFU\\MSSQLSERVER01"
database = "Malaria_DB"
#Step2: Instatiate a connection String
conn_str = (
    "Driver={ODBC Driver 17 for SQL Server};"
    f"Server={server};"
    f"Database={database};"
    "Trusted_Connection=yes;"
)
#Step3: Connect to SQL database and Append raw files to the respective in form tabl
params = urllib.parse.quote_plus(conn_str)
engine = create_engine(
    f"mssql+pyodbc:///?odbc_connect={params}",
    fast_executemany=True

                        )

df1.to_sql(
    "Malaria2023",
    engine,
    if_exists="append",
    index=False,
    chunksize=10000
)
```

```
C:\Users\User\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas\io\sq
l.py:1648: SAWarning: Unrecognized server version info '17.0.1105.2'.  Some SQL Serv
er features may not function properly.
  con = self.exit_stack.enter_context(con.connect())
```

Out[20]: -1

NOTE:

- Append: New data to the existing data.
- replace: Deletes/replaces the old data with New data
- Index=False; Deletes the index column from the dataframe before import to the DB.

In [28]:
```
#Final Inspection
df1.head()
```

Out[28]:

	orgunitlevel1	orgunitlevel2	orgunitlevel3	organisationunitid	organisationunitname	E M Tc 2 F Ja
0	MOH - Uganda	Karamoja	Abim District	NREoMszwQZW	Abim District	
1	MOH - Uganda	Acholi	NaN	SUvODYOcaVf	Acholi	
2	MOH - Uganda	West Nile	Adjumani District	QYiQ2KqgCxj	Adjumani District	
3	MOH - Uganda	Acholi	Agago District	ztIyIYAzFKp	Agago District	
4	MOH - Uganda	Lango	Alebtong District	p7EEgDEX3jT	Alebtong District	

5 rows × 487 columns

In [ ]: